Tutorial Sheet — TD 1 bis

Algorithmic Complexity and Asymptotic Notation

Learning Objectives

- Distinguish the **best**, **worst** and **average** case of an algorithm, and state which one a bound refers to.
- Count the iterations of independent, dependent and logarithmic loops, alone or combined.
- Derive higher-order complexities such as **$\Theta(n \log n)$, $\Theta(n^2 \log n)$, $\Theta(n \log^2 n)$** & **$\Theta(n^3)$** from nested loops.

Exercise 1

Consider the following two search algorithms on an array **A[1..n]**.

```
LINEAR-SEARCH(A[1..n], x):
    for i = 1 to n:
        if A[i] == x:
            return i
    return NOT-FOUND

BINARY-SEARCH(A[1..n], x):          // A is sorted
    low = 1; high = n
    while low <= high:
        mid = floor((low + high) / 2)
        if A[mid] == x:      return mid
        else if A[mid] < x:  low = mid + 1
        else:                high = mid - 1
    return NOT-FOUND
```

1. Give the **best** case and the **worst** case of LINEAR-SEARCH (in number of comparisons **A[i] == x**), with their **Θ** bounds.
2. Give the average case of LINEAR-SEARCH, assuming **x** is in the array and is equally likely to be at each of the **n** positions.
3. Give the best case and the worst case of BINARY-SEARCH (in number of loop iterations), with their **Θ** bounds.

Exercise 2

Give the tight bound **$\Theta(\cdot)$** on the number of times the line marked (*) is executed, as a function of n, for each of the following fragments.

(a)

```
for i = 1 to n:
    for j = 1 to n:
        k = 1
        while k < n:
            k = 2 * k
            count = count + 1        (*)
```

(b)

```
for i = 1 to n:
    j = i
    while j >= 1:
        j = j // 2                  (*)
```

(c)

```
for i = 1 to n:
    for j = 1 to i:
        for k = 1 to j:
            count = count + 1       (*)
```

(d) Does it matter, for these three fragments, whether we ask for the best, the worst or the average case? Justify.

Exercise 3

Part A — Insertion sort (the version below counts one comparison each time **A[j] > key** is evaluated while **j ≥ 1**).

```
INSERTION-SORT(A[1..n]):
    for i = 2 to n:
        key = A[i]; j = i - 1
        while j >= 1 and A[j] > key:
            A[j+1] = A[j]             // one shift
            j = j - 1
        A[j+1] = key
```

1. Describe the best-case input and give the exact number of comparisons and its **Θ** bound.
2. Describe the worst-case input and give the exact number of shifts and its **Θ** bound.
3. For a uniformly random permutation of n distinct values, show that the expected number of shifts is **n(n−1)/4**, and deduce the order of growth of the average case. (Hint: the number of shifts equals the number of inversions of the array.)
4. Is the statement "insertion sort runs in **Θ(n²)**" correct? Reformulate it precisely.

Part B — Give the tight bound Θ(·) on the number of executions of the line (*):

```
for i = 1 to n:
    j = 1
    while j <= n:
        j = 2 * j
        for k = 1 to i:
            count = count + 1       (*)
```

Part C — Give the tight bound Θ(·) on the number of executions of the line (*):

```
for i = 1 to n:
    a = 1
    while a < n:
        a = 2 * a
        b = 1
        while b < n:
            b = 2 * b
            count = count + 1       (*)
```